Pneumonia Detection from Chest X-Rays: Model, Explainability, and Fairness

Overview

In this assignment, you will develop a pneumonia detection system from chest X-ray images that addresses three critical aspects of machine learning in healthcare:

1. **Disease Classification**: Building a model that accurately detects pneumonia
2. **Model Explainability**: Creating techniques to explain model decisions to medical professionals
3. **Fairness Assessment**: Ensuring the model performs fairly across different demographic groups

This assignment will challenge you to balance performance, interpretability, and fairness in a real-world medical application.

Creative Freedom & Real-World Application

What makes this assignment unique is the degree of **creative freedom** you have in tackling each task. While we provide a basic framework and evaluation metrics, the specific approaches and techniques you implement are up to you. This mirrors real-world ML projects where:

- You must make design decisions based on your understanding of the problem
- There are multiple valid approaches to solving each challenge
- You need to balance competing objectives (accuracy, explainability, fairness)

- Your solutions must work within practical constraints

Unlike traditional assignments with rigid requirements, here you can explore different architectures, visualization techniques, and fairness approaches. We encourage you to think critically and apply your creativity to develop innovative solutions that address the fundamental challenges in medical AI.

Dataset

We're using the RSNA Pneumonia Detection Challenge dataset, which contains chest X-ray images labeled for pneumonia detection.

Dataset Source: RSNA Pneumonia Detection Challenge on Kaggle

Dataset Format

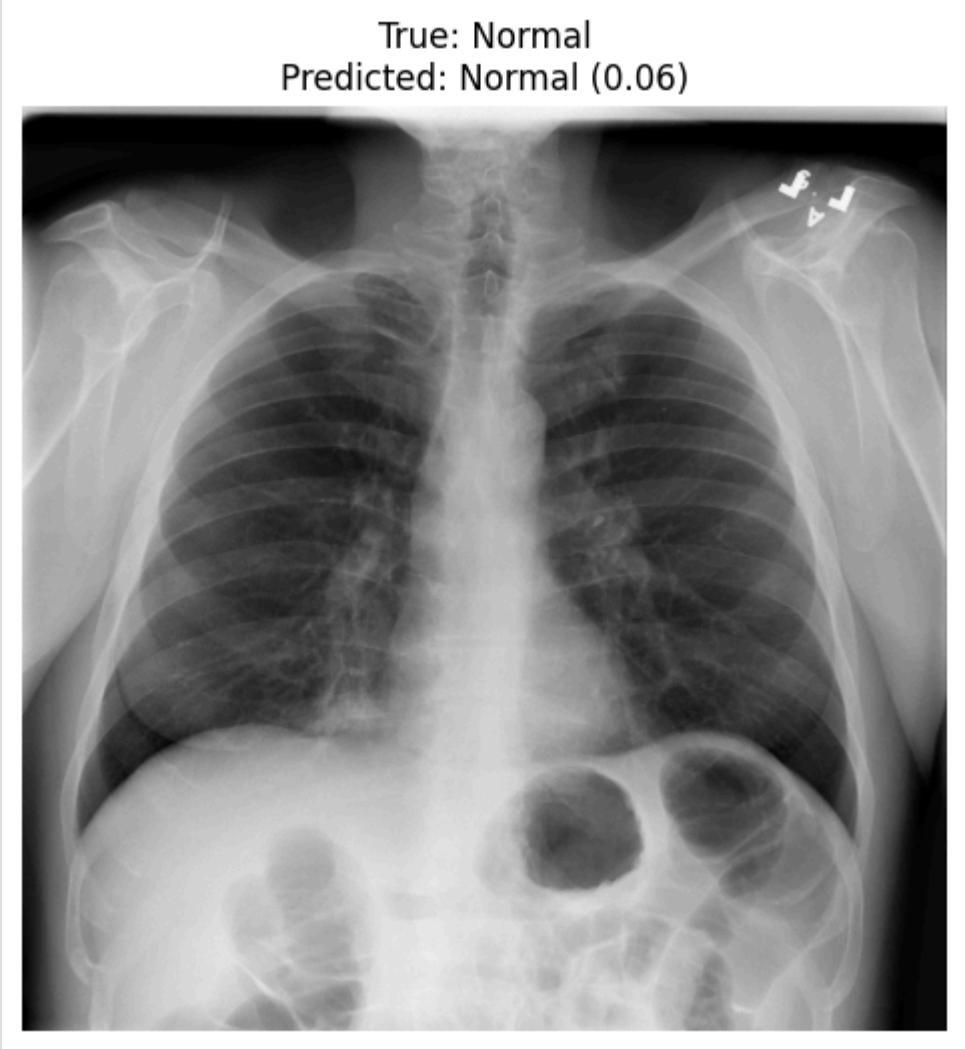
- **Images**: Chest X-ray DICOM files located in `rsna-pneumonia-detection-challenge/stage_2_train_images/`
- **Labels**: CSV file at `rsna-pneumonia-detection-challenge/stage_2_train_labels.csv`
- **Demographics**: Patient sex data is available in the DICOM files (`PatientSex` attribute)

Data Structure

The labels CSV contains the following columns: - `patientId`: Unique patient identifier matching the DICOM filename - `Target`: Binary classification (1 = pneumonia, 0 = normal) - `x`, `y`, `width`, `height`: Bounding box coordinates for pneumonia opacity (only present for positive cases)

Sample Image

Below is an example of a normal chest X-ray from the dataset:



Normal Chest X-ray

Tasks

Task 1: Pneumonia Classification (20 points)

Develop a CNN model that accurately classifies chest X-rays as normal or showing signs of pneumonia.

Requirements: - Implement the `SimplePneumoniaClassifier` class in the template - Your model should achieve a high AUC-ROC score (threshold for full points: 0.85) - Points are awarded quadratically based on your AUC-ROC score

Implementation: - Complete the model architecture in `__init__` - Implement the forward pass in `forward` - Create a prediction method in `predict` - Ensure proper checkpoint saving/loading functionality

Task 2: Importance Heatmaps for Explainability (40 points)

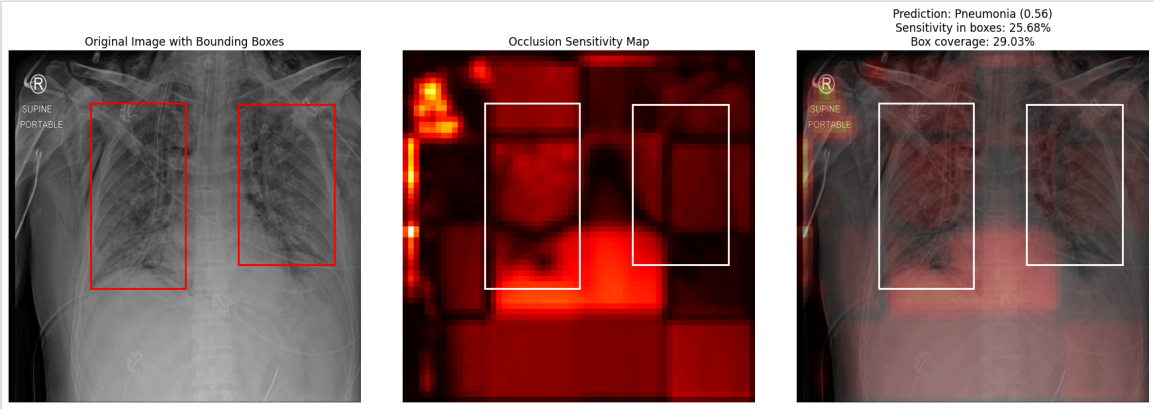
Develop a method to highlight regions in the X-ray that influenced your model's prediction.

Requirements: - Implement the `get_importance_heatmaps` function - Your heatmaps should focus on the actual pneumonia regions in positive cases - Evaluation metric: Average percentage of sensitivity inside the ground truth bounding boxes - Points are awarded quadratically based on how well your heatmaps align with actual pneumonia regions (threshold: 35%)

Implementation:

- Generate heatmaps that highlight areas most affecting the prediction
- Ensure your heatmaps are properly normalized
- The results will be assessed by checking the percentage of values in heatmap inside the bounding boxes.

The image below illustrates how the importance coverage is calculated - we measure what percentage of the total importance in your heatmap falls within the ground truth bounding boxes:



Importance Coverage Calculation

Task 3: Fairness Assessment and Mitigation (40 points)

Analyze and mitigate potential biases in your model's predictions across demographic groups.

Requirements: - Implement the `fair_predict` function - Your solution should reduce prediction disparities between male and female patients - Evaluation metrics: - Maintain AUC-ROC ≥ 0.8 - Minimize prediction rate disparity between groups (< 0.01) - Minimize TPR disparity between groups (< 0.005) - TPR (True Positive Rate) is the proportion of actual positive cases correctly identified by the model. TPR disparity means different demographic groups have different rates of correct pneumonia detection.

Implementation: - Analyze potential biases in the base model's predictions - Implement group-specific thresholds to balance fairness criteria - Ensure fairness improvements don't significantly reduce overall performance

Submission Requirements

Code Structure

You must submit a single Python file following the template structure in `student_template.py`. Your submission must include:

1. `SimplePneumoniaClassifier` class with:
    - `__init__`: Model architecture
    - `forward`: Forward pass
    - `load_checkpoint`: Loading model weights
    - `predict`: Inference function
2. `get_importance_heatmaps` function for explainability
3. `fair_predict` function for fairness improvements

Model Weights Format

IMPORTANT: Your model weights must be saved in a specific format for proper loading by the grader:

```
# Saving model weights (example code)
checkpoint = {
    'epoch': epoch,
    'model_state_dict': model.state_dict(),  # MUST use this
        key name
    'optimizer_state_dict': optimizer.state_dict(),
    # You can include other metadata if needed
```

```
}
torch.save(checkpoint, os.path.join(checkpoint_dir,
        'best_model.pt'))
```

The grader will load your model using:

```
checkpoint = torch.load('checkpoints/best_model.pt')
model.load_state_dict(checkpoint['model_state_dict'])
```

Key requirements: - Save your checkpoint in the `checkpoints/` directory (it will be created by your model) - Name your best model file `best_model.pt` - Include the model state dictionary with the key `'model_state_dict'`

Grading Criteria

Task	Max Points	Evaluation
Pneumonia Classification	20	AUC-ROC ≥ 0.85 for full points
Importance Heatmaps	40	≥ 35% importance coverage in pneumonia regions for full points
Fairness Assessment	40	Points split evenly between AUC-ROC maintenance, prediction rate parity, and TPR parity

Note: For tasks where you don't meet the threshold, points are awarded quadratically based on your score relative to the threshold (stricter penalty for lower scores).

Common Issues to Avoid

1. **Model loading issues:**
   - Ensure your checkpoint contains the model weights with the key `'model_state_dict'`
   - Don't change the expected model class name or function signatures

2. **Tensor shape problems:**
   - Handle different input formats (tensor/numpy array) and dimensions correctly
   - Make sure output shapes match the expected format
3. **Fairness implementation:**
   - Don't compromise overall performance too much for fairness
   - Carefully balance the different fairness metrics
4. **Path issues:**
   - Use relative paths for accessing files within your directory
   - Don't hardcode absolute paths in your code

Testing Your Submission

Before submitting, verify that:

1. Your code runs without errors when instantiating all classes and functions
2. Your model can be loaded using the `load_checkpoint` method
3. Your `get_importance_heatmaps` function works with various input formats
4. Your `fair_predict` function correctly handles the sex attribute parameter

FAQ

Q: Can I use pre-trained models? A: Yes, but make sure to include only the necessary components and document any external weights.

Q: How should I handle the variable image sizes? A: Consider using adaptive pooling or resizing to handle different dimensions.

Q: What if I don't have a GPU for training? A: The model will be evaluated on GPU/MPS if available. Your code should be device-agnostic.

Q: How do I balance fairness and performance? A: Start by analyzing where biases occur.

Good luck with your assignment! Remember that real-world ML applications in healthcare require balancing multiple objectives beyond just accuracy.